2520030117
skill-07

EXPERIMENT – 7

Title:

Process Synchronization and Child Process Monitoring Using `waitpid()`

Aim:

To implement process synchronization using `waitpid()`, monitor child processes, retrieve the `PATH` variable, locate executable commands, verify permissions, and handle missing commands.

Objective:

- To understand process synchronization.
- To monitor child processes.
- To use `waitpid()` for synchronization.
- To retrieve the `PATH` environment variable.
- To search directories in `PATH`.
- To locate executable files.
- To verify file permissions.
- To handle missing commands.
- To test command resolution.

Theory:

`waitpid()` is used by a parent process to wait for a specific child process to finish.

The `PATH` environment variable contains a list of directories where executable programs are located.

For example, when the user enters:

ls

the shell searches the directories in `PATH` to find the executable.
The shell checks whether the command exists and whether it has execute permission. If the command is not found, an error message is displayed.

Algorithm:

1. Start the shell.
2. Retrieve the `PATH` environment variable.
3. Read the command entered by the user.
4. Parse the directories stored in `PATH`.
5. Search each directory for the command.
6. Check whether the executable exists.
7. Check whether the file has execute permission.
8. Create a child process using `fork()`.
9. Execute the command using `execvp()`.
10. Use `waitpid()` in the parent process to wait for the child.
11. Handle missing commands and execution errors.
12. Repeat until the user enters `exit`.
13. Stop.

Functions Used:

Function	Purpose
`getenv( )`	
`strtok( )`	Check file and execute permissions
`access( )`	
Retrieve `PATH`	
	Parse directories
`fork()`	Create child process
`execvp( )`	for a specific
`waitpid ()`	child Display
`printf( )`	
Execute	output
command Wait	`perror( )`
	Display errors

Sample Input:

2520030117_vaishali> echo hello
2520030117_vaishali> pwd

2520030117_vaishali> ls
2520030117_vaishali> xyzabc

Sample Output:

hello
/home/vaishali
...
Command not found

Observation:

1. The `PATH` variable was retrieved successfully.
2. Commands were searched in the PATH directories.
3. Executable files were located successfully.
4. File permissions were checked.
5. Child processes were created successfully.
6. The parent process waited for the child using `waitpid()`.
7. Missing commands were handled correctly.

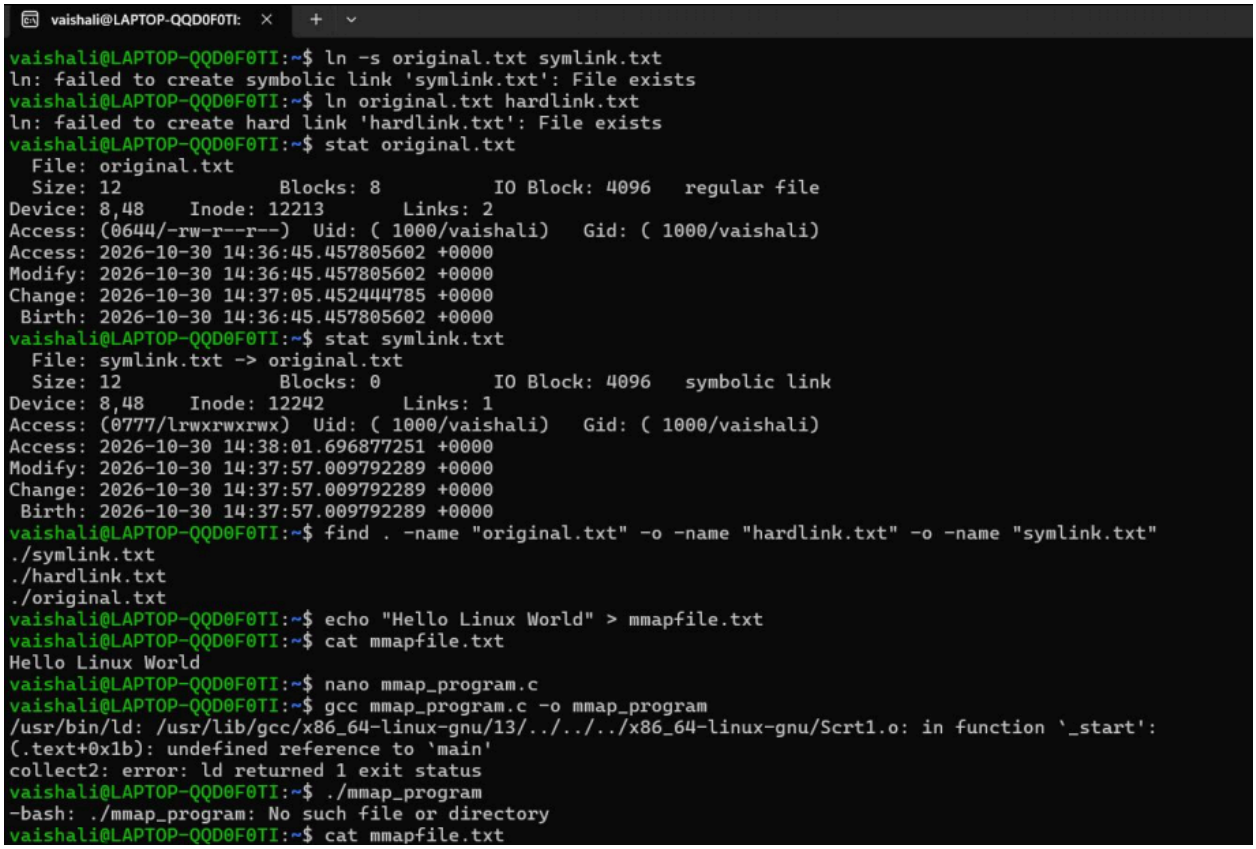
Result:

The shell successfully performed **PATH-based command resolution and child process synchronization using `waitpid()`**.

Conclusion:

The experiment successfully demonstrated **child process monitoring and synchronization using `waitpid()`**. The shell also retrieved the PATH variable, searched directories, checked executable permissions, and handled missing commands.




vaishali@LAPTOP-QQD0F0TI:
vaishali@LAPTOP-QQD0F0TI:~$ ln -s original.txt symlink.txt
ln: failed to create symbolic link 'symlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
ln: failed to create hard link 'hardlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ stat original.txt
File: original.txt
Size: 12 Blocks: 8 IO Block: 4096 regular file
Device: 8,48 Inode: 12213 Links: 2
Access: (0644/-rw-r--r--) Uid: (1000/vaishali) Gid: (1000/vaishali)
Access: 2026-10-30 14:36:45.457805602 +0000
Modify: 2026-10-30 14:36:45.457805602 +0000
Change: 2026-10-30 14:37:05.452444785 +0000
Birth: 2026-10-30 14:36:45.457805602 +0000
vaishali@LAPTOP-QQD0F0TI:~$ stat symlink.txt
File: symlink.txt -> original.txt
Size: 12 Blocks: 0 IO Block: 4096 symbolic link
Device: 8,48 Inode: 12242 Links: 1
Access: (0777/lrwxrwxrwx) Uid: (1000/vaishali) Gid: (1000/vaishali)
Access: 2026-10-30 14:38:01.696877251 +0000
Modify: 2026-10-30 14:37:57.009792289 +0000
Change: 2026-10-30 14:37:57.009792289 +0000
Birth: 2026-10-30 14:37:57.009792289 +0000
vaishali@LAPTOP-QQD0F0TI:~$ find . -name "original.txt" -o -name "hardlink.txt" -o -name "symlink.txt"
./symlink.txt
./hardlink.txt
./original.txt
vaishali@LAPTOP-QQD0F0TI:~$ echo "Hello Linux World" > mmapfile.txt
vaishali@LAPTOP-QQD0F0TI:~$ cat mmapfile.txt
Hello Linux World
vaishali@LAPTOP-QQD0F0TI:~$ nano mmap_program.c
vaishali@LAPTOP-QQD0F0TI:~$ gcc mmap_program.c -o mmap_program
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/13/../../../x86_64-linux-gnu/Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
vaishali@LAPTOP-QQD0F0TI:~$ ./mmap_program
-bash: ./mmap_program: No such file or directory
vaishali@LAPTOP-QQD0F0TI:~$ cat mmapfile.txt